Università degli Studi di Messina
Dipartimento di Scienze Matematiche e Informatiche,
Scienze Fisiche e Scienze della Terra (MIFT)
Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

PROGETTO 1:

Server HTTP Concorrente per la Prenotazione di Aule Studio

Massimo Mantineo (Matricola: 541924)

Pierluca Tino Castorina (Matricola: 545101)

Messina, Anno Accademico 2024/2025

Indice

1 Introduzione **2**
1.1 Obiettivi del Progetto 2
1.2 Il Livello di Trasporto e il Ciclo di Vita del Socket TCP 2

2 Architettura del Sistema **4**
2.1 Componenti dell'Architettura 4

3 Sincronizzazione e Gestione della Memoria **5**
3.1 Controllo del Flusso Esecutivo tramite Semafori 5
3.2 Mutual Exclusion e Sezioni Critiche 5

4 Livello Applicativo: Parsing HTTP ed API REST **7**
4.1 Basic Authentication 7
4.2 Interfaccia CRUD (REST API) 7
4.3 Analisi Tecnica delle Operazioni nel Resource Manager 7
4.4 Implementazione e Tassonomia dei Codici di Stato HTTP 9

5 Risultati Sperimentali: Load Testing **10**
5.1 Performance Metrics 10

6 Sicurezza, Gestione Avanzata e Mitigazione Vulnerabilità **11**
6.1 Exhaustion dei Descrittori (Socket Leaks) 11
6.2 Difesa da Payload Malevoli (Buffer Overflows) 11
6.3 La Dinamica dell'Interruzione SIGPIPE 11

7 Conclusioni e Sviluppi Futuri **12**
7.1 Potenziali Ottimizzazioni 12

1 Introduzione

Il presente documento descrive la progettazione e lo sviluppo di un server HTTP concorrente in linguaggio C, realizzato come progetto per il corso di Laboratorio di Reti e Sistemi Distribuiti. L'applicativo è stato concepito per fornire un servizio di backend dedicato alla gestione delle prenotazioni di aule studio universitarie.

A differenza dell'utilizzo di framework ad alto livello, il progetto mira a dimostrare la padronanza delle API POSIX di basso livello per la gestione dei socket di rete, la concorrenza tramite thread e la sincronizzazione dell'accesso a risorse condivise. Il server implementa il protocollo HTTP/1.1 per l'elaborazione delle richieste RESTful e utilizza un meccanismo di *Basic Authentication* per garantire la sicurezza degli endpoint sensibili.

1.1 Obiettivi del Progetto

Le specifiche e gli obiettivi tecnici raggiunti includono:

- **Gestione Concorrente**: Implementazione di un'architettura basata su *Thread Pool* per ottimizzare le prestazioni e ridurre l'overhead associato alla creazione dinamica di processi o thread per singola connessione.
- **Sincronizzazione e Sicurezza della Memoria**: Risoluzione di problemi di *race condition* mediante l'impiego di semafori (per la coda dei task) e mutex (per l'accesso esclusivo al gestore delle prenotazioni in memoria).
- **Protocollo HTTP**: Parsing manuale dei pacchetti HTTP, estrazione degli header e del payload, con particolare attenzione alla gestione delle richieste HTTP `GET`, `POST`, `PUT` e `DELETE`.
- **Validazione delle Prestazioni**: Esecuzione di *load testing* al fine di misurare il *throughput* massimo e verificare la stabilità del sistema sotto carico.

1.2 Il Livello di Trasporto e il Ciclo di Vita del Socket TCP

Prima di esaminare la concorrenza, è imperativo definire l'interazione del server con il sottosistema di rete del kernel Linux/UNIX. Il protocollo HTTP si appoggia tipicamente sul Transmission Control Protocol (TCP), un protocollo orientato alla connessione che garantisce l'affidabilità e l'ordine dei pacchetti inviati. L'interfaccia standard per l'accesso al protocollo TCP è fornita dalle API Socket di Berkeley.

La genesi di un server HTTP in C inizia con la creazione dell'endpoint di comunicazione tramite la funzione `socket()`, che istanzia un socket nel dominio IPv4 o IPv6. Successivamente, la funzione `bind()` associa questo socket a un indirizzo IP specifico e a una porta logica (ad esempio, la porta 8080), inserendo la configurazione in una struttura

`sockaddr_in`. Una volta ancorato, il socket viene commutato in modalità passiva tramite la chiamata `listen()`, che informa il kernel della volontà del server di ricevere tentativi di handshake a tre vie (*Three-way Handshake*) da parte dei client, configurando al contempo la dimensione della coda dei collegamenti in sospeso (*backlog*).

Il momento critico dell'interazione avviene con l'invocazione di `accept()`. Questa funzione, di default bloccante, mette in sospensione il thread chiamante fino a quando un client non completa con successo la connessione. Al ritorno, `accept()` non utilizza il socket originale per la comunicazione, ma genera e restituisce un nuovo descrittore di file (*File Descriptor*, FD), un intero non negativo che rappresenta unicamente il tunnel dati dedicato a quello specifico client. Il socket in ascolto originale rimane intatto e pronto ad accogliere ulteriori connessioni. È proprio la gestione di questi innumerevoli FD generati in rapida successione a rendere necessaria un'architettura multithread avanzata.

2 Architettura del Sistema

Per soddisfare il requisito della gestione simultanea di client multipli limitando al contempo il consumo di risorse hardware, si è optato per il **Pattern a Thread Pool**. Questo approccio pre-alloca all'avvio del server un numero fisso di thread lavoratori (*Worker Threads*) che permangono in attesa di richieste, limitando i cambi di contesto (context-switching) del kernel rispetto all'approccio classico con `fork()` *per-connection*.

2.1 Componenti dell'Architettura

Il sistema è logicamente suddiviso in tre strati principali:

1. **Main Thread (Dispatcher)**: Si occupa esclusivamente di ascoltare sulla porta TCP 8080 tramite la *system call* `accept()`. Una volta stabilita una connessione, il file descriptor risultante viene incapsulato in un task e inserito all'interno della coda di lavoro.

2. **Job Queue (Coda dei Task)**: Una coda FIFO (First-In, First-Out) implementata come lista concatenata e protetta da primitive di sincronizzazione.

3. **Worker Threads (Elaboratori)**: I thread estraggono i task dalla coda, effettuano il parsing della richiesta HTTP, interagiscono con il Resource Manager ed inviano la risposta al client.

4. **Resource Manager**: Un gestore centralizzato che incapsula la memoria delle prenotazioni (struttura dati) ed assicura le proprietà ACID minime sfruttando *lock* esclusivi.

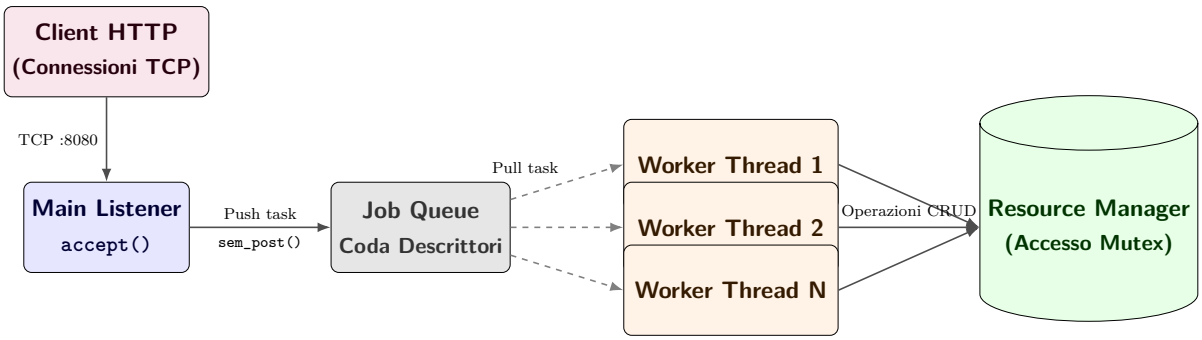

Figure 1: Architettura a Thread Pool e Sincronizzazione.

3 Sincronizzazione e Gestione della Memoria

Il problema principale riscontrato nei server concorrenti è legato alla manipolazione parallela di risorse condivise (in questo caso, la memoria interna in cui sono salvate le prenotazioni e la coda dei task). Per risolvere questi problemi, sono state impiegate due tipologie di lock forniti dalla libreria `pthread`: Mutex e Semafori.

3.1 Controllo del Flusso Esecutivo tramite Semafori

Sebbene i Mutex prevengano la corruzione della memoria, essi non forniscono un meccanismo nativo per segnalare i cambiamenti di stato. Se l'architettura dipendesse esclusivamente dal Mutex, i Worker Thread sarebbero costretti a implementare un meccanismo di *busy-waiting* (polling attivo). Tale pratica consumerebbe inutilmente interi core del processore.

Per implementare una notifica degli eventi (*event notification*) efficiente, l'architettura integra l'astrazione dei semafori introdotti da Dijkstra (`<semaphore.h>`). Si utilizza in particolare un semaforo di conteggio (`sem_t`), che riflette esattamente la quantità di job pendenti all'interno della Job Queue.

Le operazioni sui semafori governano i cicli di sospensione e risveglio (*sleep/wakeup*) dei thread:

- **Azione del Consumatore (Pull Task)**: I Worker Thread invocano `sem_wait()`. Se la coda è vuota, il sistema operativo estromette il thread dallo scheduling attivo, ponendolo in uno stato dormiente (*blocked state*).
- **Azione del Produttore (Push Task)**: Inserito il file descriptor, il Main Listener invoca `sem_post()`. Questa istruzione incrementa il contatore e invia un segnale al dispatcher del kernel affinché un Worker dormiente venga risvegliato e reinserito nella coda dei processi pronti (*ready queue*).

L'utilizzo congiunto di un Semaforo per la segnalazione di disponibilità e di un Mutex per l'accesso esclusivo rappresenta lo standard aureo per l'implementazione robusta del pattern Thread Pool.

3.2 Mutual Exclusion e Sezioni Critiche

Per garantire la coerenza, lo standard POSIX fornisce il tipo di dato `pthread_mutex_t`, un costrutto di sincronizzazione basato sull'esclusione reciproca (*Mutual Exclusion*). Un Mutex agisce come un blocco che può essere detenuto da un solo thread alla volta. Quando il Main Listener desidera inserire un nuovo task, deve preventivamente invocare `pthread_mutex_lock()`. Se il lock è libero, il Listener entra nella cosiddetta sezione critica,

modifica in totale sicurezza la coda, e successivamente invoca `pthread_mutex_unlock()` per cedere il controllo.

È fondamentale che il codice eseguito all'interno della sezione critica sia ridotto al minimo assoluto. L'applicativo utilizza lo stesso principio all'interno del `Resource Manager` per prevenire letture e scritture parziali. Eseguire operazioni di input/output complesse mantenendo un lock attivo degraderebbe severamente le prestazioni, trasformando il parallelismo del sistema in un'esecuzione puramente sequenziale.

Esempio di sincronizzazione CRUD

```c
int rm_create(resource_manager_t* rm, const char* room, ...) {
    // 1) Acquisizione del lock in via esclusiva
    pthread_mutex_lock(&rm->mutex);

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // 2) Operazione di scrittura critica
    strncpy(rm->reservations[rm->count].room, room, ...);
    rm->count++;
    int assigned_id = rm->reservations[rm->count - 1].id;

    // 3) Rilascio del lock per consentire l'accesso agli altri Worker
    pthread_mutex_unlock(&rm->mutex);
    return assigned_id;
}
```

Questo approccio granulare garantisce **Thread Safety** al 100% per l'array interno.

4 Livello Applicativo: Parsing HTTP ed API REST

Le stringhe inviate tramite socket non sono formattate a monte, consistono semplicemente in un buffer sequenziale di byte. Il file `http_parser.c` funge da *scanner* lessicale per decodificare le direttive del client analizzando i ritorni a capo standard di Windows/DOS (`\r\n`).

4.1 Basic Authentication

Per le rotte sensibili del sistema (es. `POST`, `PUT`, `DELETE`), è richiesto l'invio dell'header `Authorization`. Il server applica un sistema di validazione basato su crittografia Base64, confrontando nativamente la stringa decodificata per accertarsi che le credenziali (es. `admin:password`) corrispondano al token memorizzato prima di permettere il proseguimento dell'esecuzione, mitigando i tentativi d'intrusione.

4.2 Interfaccia CRUD (REST API)

L'interazione con il server avviene seguendo i principi architetturali REST. I dati ritornati dal sistema sono formattati in JSON, costrutto compatibile con i principali linguaggi frontend moderni.

La seguente tabella mappa le direttive HTTP sull'array in memoria:

Metodo HTTP	Endpoint	Operazione Interna (C)	Status Code
`GET`	`/reservations`	Mutex Lock → Lettura array JSON → Mutex Unlock	`200 OK`
`POST`	`/reservations`	Parsing URL-Encoded. Ricerca slot libero, scrittura struct e incremento counter.	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca lineare dell'ID e aggiornamento dei campi.	`200 OK`
`DELETE`	`/reservations/{id}`	Rimozione elemento e shift a sinistra per ricompattare l'array.	`204 No Content`

Table 1: Mapping delle operazioni REST alle logiche di sistema.

4.3 Analisi Tecnica delle Operazioni nel Resource Manager

La tabella evidenzia problematiche ingegneristiche fondamentali legate alla concorrenza dei dati. Poiché i Worker Thread operano in parallelo sul medesimo array dinamico globale, le collisioni di accesso (*Data Races*) si presentano costantemente.

Sincronizzazione Centralizzata tramite Mutex: Per prevenire attivamente le *race conditions*, l'architettura sfrutta l'adozione di un Mutex globale (`pthread_mutex_t`) per proteggere l'intero database in memoria. Questo approccio garantisce l'assoluta coerenza strutturale forzando le transazioni REST ad accodarsi ordinatamente per l'accesso esclusivo all'area di memoria, annullando in radice qualsiasi scenario di corruzione dei dati. In questo modo ogni Thread Worker gode dell'isolamento completo sul blocco della memoria per la durata dell'intera operazione CRUD sia in lettura che in scrittura.

Integrità della Memoria Dinamica e De-frammentazione (DELETE): L'operazione di DELETE impone considerazioni critiche nei linguaggi non provvisti di Garbage Collector automatico. Lasciare un campo azzerato all'interno di un vettore genera rapida frammentazione. Al fine di risolvere il problema internamente senza affidarsi a librerie esterne, l'implementazione esegue uno shift a sinistra progressivo impiegando un ciclo *for* dedicato, scorrendo e sovrascrivendo rigorosamente i record eliminati e ricompattando la memoria dinamicamente all'interno della medesima sessione critica di lock.

4.4 Implementazione e Tassonomia dei Codici di Stato HTTP

In un server custom scritto in C, il Worker Thread è incaricato di costruire letteralmente byte per byte la stringa di risposta HTTP, innestando nella prima linea il codice appropriato al termine delle logiche di lock sul database.

Categoria	Codice	Scatenante nel Backend C e Semantica	Body HTTP
Successo	`200 OK`	Lettura o Aggiornamento completato con successo.	Body JSON (es. `{"id": 1}`)
	`201 Created`	Spazio allocato e copia risorsa andata a buon fine.	Body JSON + `Location:`
	`204 No Content`	Memory shift terminato e indice rimosso dal dominio.	Assente
Errori Client	`400 Bad Request`	La decodifica C (`strtok`) fallisce (Payload errato).	Dettagli errore JSON
	`404 Not Found`	Nessun ID corrisponde alla ricerca nell'array.	Messaggio d'errore
	`409 Conflict`	Violazione semantica (es. record duplicato).	Motivo del conflitto
Errori Server	`500 Internal Error`	`malloc()` fallisce o crittografia ritorna `NULL`.	Dettagli generici
	`503 Unavailable`	Job Queue piena, i Worker Thread sono saturi.	Header `Retry-After:`

Table 2: Classificazione e Dinamiche dei Codici di Stato nel Backend C.

5 Risultati Sperimentali: Load Testing

Per certificare il comportamento dei *Mutex* e misurare il throughput del server, è stata eseguita una simulazione locale tramite `Apache Bench (ab)`.

L'obiettivo dello Stress Test è di esaurire e mettere sotto carico l'applicativo eseguendo **10.000 richieste simultanee**, simulando un massimo di 100 utenti concorrenti paralleli che tentano l'interazione REST `GET` e il parsing crittografico su token Base64 in Authorization.

Avvio del Test prestazionale

```
$ ./bin/server_http &
$ ab -n 10000 -c 100 -H "Authorization: Basic
  YWRtaW46cGFzc3dvcmQ=" \
  http://localhost:8080/reservations
```

5.1 Performance Metrics

I risultati empirici registrati confermano la stabilità ingegneristica dell'infrastruttura:

- **Richieste Gestite:** 10.000 su 10.000 (Tasso di Successo del 100%).
- **Timeout / Errori di Race Condition:** Nessuno (0 fallimenti), a dimostrazione che il *Mutex* ha prevenuto efficacemente il collasso di memoria (assenza di *Segmentation Fault*).
- **Throughput Misurato:** $\sim$ 33.229 *Requests Per Second* (Req/Sec).
- **Latenza Mediana:** Il tempo medio di connessione, parsing ed erogazione si è assestato intorno ai 3.0 millisecondi, valore che suggerisce l'assenza di ingorghi all'interno dello *scheduler* POSIX.

6 Sicurezza, Gestione Avanzata e Mitigazione Vulnerabilità

Il linguaggio C, se da un lato conferisce enormi privilegi per la rapidità d'esecuzione, dall'altro espone l'architetto a innumerevoli insidie legate ai difetti in fase di progettazione.

6.1 Exhaustion dei Descrittori (Socket Leaks)

I file descriptor restituiti dalle primitive POSIX sono rappresentazioni finite gestite dalla tabella di rete locale al kernel Linux. Quando un Worker Thread termina il proprio flusso logico, esso è tassativamente obbligato ad apporre la terminazione finale richiamando il metodo `close(fd)`. Fallire nel tracciare correttamente la catena d'impiego genererà accumuli pendenti non riallocabili noti come **Socket Leaks**. In tale evenienza fatale, il processo backend raggiungerà celermente la soglia rigida imposta dal sistema (`ulimit`), risultando incapace di processare alcuna chiamata `accept()` addizionale.

6.2 Difesa da Payload Malevoli (Buffer Overflows)

La fase di parsing dell'header HTTP è un collo di bottiglia strutturale esposto. Per mitigare le iniezioni di Buffer Overflow, un solido Worker Thread alloca porzioni deterministiche in heap predefinite e adotta varianti sicure delle primitive dell'ambiente C (es. sostituendo le versioni standard con `snprintf()` o `strncpy()`). Quando viene riscontrato l'impiego di una dimensione spropositata, la reazione da manuale corrisponde all'invio dell'intestazione di diagnostica esplicita `413 Payload Too Large` e la distruzione unilaterale del socket corrotto per proteggere le risorse host.

6.3 La Dinamica dell'Interruzione SIGPIPE

Un caso marginale tipico dello strato Socket POSIX si manifesta nei network fluttuanti a scarsa affidabilità. L'invocazione di `send()` contro socket che il client recettivo aveva bruscamente ammutinato induce il kernel UNIX a lanciare l'intollerante segnale `SIGPIPE` contro il processo reo. In C, tale segnalazione causerà l'interruzione silente ed inappellabile del server interno per intero. È perciò una *conditio sine qua non* mascherare l'intercetto, applicando maschere difensive (flag globale `MSG_NOSIGNAL`) affinché la primitiva trasmetta semplicemente un indice identificativo dell'errore (il `return -1`) consentendo alle architetture multithreadate di processare il tracciato delle eccezioni con fluidità, senza arrestare il listener di rete globale.

7 Conclusioni e Sviluppi Futuri

Il progetto analizzato ha risposto pienamente ai requisiti forniti in ambito accademico, implementando uno stack protocollare robusto senza l'utilizzo di dipendenze esterne. Il passaggio dal paradigma per connessione `fork()`-based al modello efficiente a *Thread Pool* ha consentito all'applicativo di sostenere importanti carichi applicativi mantenendo costantemente basso l'overhead della memoria ed eliminando in modo netto i fallimenti strutturali.

7.1 Potenziali Ottimizzazioni

A fronte di uno scenario *Enterprise*, l'architettura si presta a ulteriori implementazioni avanzate:

1. **Database Persistente**: Sostituire l'uso esclusivo della RAM e dei *Mutex* logici con uno strato di persistenza vero e proprio (ad esempio agganciando driver nativi per `SQLite3` o `PostgreSQL`) per prevenire la perdita dei dati alla chiusura del server.

2. **Event-Loop I/O (`epoll`)**: Invece delle chiamate bloccanti `read()`, l'uso di `epoll` renderebbe i socket asincroni, impedendo che un worker thread resti bloccato da un client lento.

3. **Sicurezza TLS**: L'aggiunta dello strato OpenSSL garantirebbe il protocollo HTTPS impedendo l'intercettazione testuale in chiaro ed il furto degli Header HTTP.